

Smart Notes — Document Q&A System

Retrieval-Augmented Generation pipeline built on Next.js, OpenAI embeddings, and Pinecone · 3-week build plan

01 System Overview

Smart Notes lets a user upload a document and ask questions about it in plain language, getting back answers grounded in that document's actual content — with the source cited — instead of a generic LLM guess. The system has two independent pipelines: an **ingestion pipeline** (runs once per uploaded document) and a **query pipeline** (runs on every question asked).

02 Ingestion Pipeline — What Happens on Upload



03 Query Pipeline — What Happens on Every Question



Why this beats asking the LLM directly: without retrieval, the model can only answer from what it memorized during training — it has never seen your document and will either refuse or hallucinate. Retrieval grounds every answer in text that was actually pulled from the source, and citations let the user verify it.

04 Tech Stack

FRONTEND Next.js 14 + TypeScript	EMBEDDINGS OpenAI text-embedding-3-small	VECTOR STORE Pinecone (free tier)	GENERATION Claude / OpenAI API
AUTH Firebase Auth	DEPLOYMENT Vercel	FILE PARSING pdf-parse	STYLING Tailwind CSS

05 Build Timeline

WEEK	GOAL	DELIVERABLE
Week 1	Core ingestion pipeline	Upload a document, chunk it, generate embeddings, and store vectors in Pinecone. No retrieval or UI polish yet — verify the pipeline works end to end.
Week 2	Retrieval + generation	Embed the user's question, run similarity search, pass top chunks to the LLM, return a grounded answer with a source citation for each claim.
Week 3	Real product UI	Document library, streaming chat interface, per-user data isolation (multi-tenancy), and proper error handling for bad uploads and empty results.
Week 4	Deploy + instrument	Ship to Vercel, add latency logging and a manual accuracy check against 10–15 test questions, then write the quantified resume bullet.

06 Key Design Decisions

- No LangChain** — every step (embed, store, retrieve, generate) is a direct API call you write yourself, so you can explain exactly how retrieval works in an interview.
- Pinecone over pgvector** — a simpler API for a first RAG build; the underlying concept (nearest-neighbor vector search) transfers directly if you swap to pgvector later.
- Overlapping chunks (100-char overlap)** — prevents a sentence from being split across two chunks and losing meaning at the boundary.
- Citations returned with every answer** — forces the system to be honest about what it actually retrieved, and is the detail that signals you understand why RAG reduces hallucination.

